HARVEST BOX

AI-Powered Crop Health Monitoring System

Official Technical Documentation

Version 2.0.0 | Production Grade

Backend: Spring Boot 3.2 | ML: FastAPI + PyTorch | Frontend: React 19

 © Harvest Box — All Rights Reserved

1. System Overview

Harvest Box is a production-grade, full-stack agricultural intelligence platform designed for Indian grape farmers, agronomists, and agri-tech operators. The system accepts a photograph of a grape leaf, runs it through a fine-tuned DenseNet121 deep learning model, assesses real-time weather-driven disease pressure, generates a comprehensive AI-authored agronomic report, and stores everything in AWS S3 and PostgreSQL for persistent history and analytics.

1.1 High-Level Architecture

The platform is composed of three independently deployable services:

- Backend — Spring Boot 3.2.1 REST API (Java 21) handling authentication, business logic, database persistence, and S3 image storage.
- ML Service — FastAPI microservice (Python 3.11+) responsible for DenseNet121 inference, weather fetching, LLM report generation, and PDF creation.
- Frontend — React 19 Single Page Application (Vite 7) delivering the farmer-facing dashboard, field setup, analytics, and user profile pages.

1.2 Request Flow

A complete end-to-end leaf analysis follows this sequence:

- Farmer uploads leaf image from the FieldSetup page (React).
- Frontend calls POST /api/v1/images/upload (Spring Boot) with the file, grid coordinates, field name, and optional GPS coordinates.
- ImageController validates the request and delegates to PlantService.
- PlantService stores the image in AWS S3 via ImageStorageService.
- PlantService calls MLClientService, which POSTs to /api/v1/analyze-field on the FastAPI ML service.
- ML Service runs DenseNet121 inference, fetches weather from Open-Meteo, generates a 9-section report via Gemini/HuggingFace, renders a multi-chart PDF, and uploads the PDF to S3.
- ML Service returns a flat JSON response with pdf_url, prediction, confidence, severity, and weather data.
- PlantService maps the ML response to a Plant entity and saves to the database.
- Frontend receives the PlantResponse DTO and displays the result; a link to the PDF report is shown.

 © Harvest Box — All Rights Reserved

2. Backend Service (Spring Boot)

The backend is a stateless REST API built with Spring Boot 3.2.1 and Java 21. It handles all user-facing API calls, JWT authentication, database persistence, S3 image storage, weather proxying, and ML service communication.

2.1 Technology Stack

Component	Technology	Version
Framework	Spring Boot	3.2.1
Language	Java	21
Database (dev)	H2 (in-memory)	Embedded
Database (prod)	PostgreSQL	Latest
Migrations	Flyway	Managed
ORM	Spring Data JPA / Hibernate	Included
Security	Spring Security + JWT	JJWT 0.12.3
Cloud Storage	AWS SDK v2 S3	2.25.16
Email	Spring Mail (SMTP)	Included
Rate Limiting	Bucket4j	8.7.0
API Docs	SpringDoc OpenAPI	2.3.0
Build Tool	Maven	Wrapper included

2.2 Module Structure

Package root: com.capstone.project

Package	Contents	Responsibility
config	AppConfig, AuthFilter, CorsConfig, JacksonConfig, OpenApiConfig, RateLimitFilter, RequestLoggingFilter, S3Config, SecurityConfig	Cross-cutting infrastructure: security chain, CORS, JWT filter, S3 client, rate limiter
controller	AuthController, ImageController, PlantController, UserController, WeatherController	REST endpoints — thin, delegate entirely to services
dto	AuthResponse, LoginRequest, MLAnalysisResponse, PlantResponse, RegisterRequest	Request/Response data transfer objects
exception	ApiException, GlobalExceptionHandler	Typed HTTP exceptions and @ControllerAdvice handler

 © Harvest Box — All Rights Reserved

model	User, Plant, VerificationToken	JPA entities mapped to database tables
repository	PlantRepository, UserRepository, VerificationTokenRepository	Spring Data JPA interfaces
service	AuthService, EmailService, ImageStorageService, JwtService, MLClientService, PlantService, UserService	All business logic

2.3 Controllers

2.3.1 AuthController — /api/v1/auth

All endpoints are public (no JWT required). Thin controller; all logic is delegated to AuthService.

Method	Path	Body / Params	Response
POST	/register	RegisterRequest {name, email, password}	AuthResponse {token: null, message: 'OTP sent'}
POST	/login	LoginRequest {email, password}	AuthResponse {user, token (JWT)}
POST	/verify-otp	{email, otp}	AuthResponse {user, token (JWT)}
POST	/resend-otp	{email}	{message: 'OTP resent'}

2.3.2 ImageController — /api/v1/images

Requires Bearer JWT. Handles multipart image upload and triggers the full analysis pipeline.

Method	Path	Parameters	Response
POST	/upload	file (MultipartFile), row (int), col (int), fieldName (String, optional), lat (Double, optional), lon (Double, optional)	PlantResponse with all analysis results including PDF URL

2.3.3 PlantController — /api/v1/plants

Requires Bearer JWT. Paginated access to analysis reports.

Method	Path	Params	Response
GET	/	page (default 0), size (default 20)	Page<PlantResponse> sorted by analyzedAt DESC
POST	/	Plant body	PlantResponse (manual creation)

 © Harvest Box — All Rights Reserved

2.3.4 UserController — /api/v1/user

Requires Bearer JWT. User profile and dashboard summary.

Method	Path	Body	Response
GET	/summary	(none)	{id, name, email, location, reportsGenerated, healthyFields, warnings}
PUT	/profile	{name?, location?}	{id, name, email, location}

2.3.5 WeatherController — /api/v1/weather

Requires Bearer JWT. Server-side proxy for OpenWeatherMap — the API key never leaves the server.

Method	Path	Params	Response
GET	/	lat, lon (doubles)	OpenWeatherMap current weather JSON
GET	/forecast	lat, lon (doubles)	OpenWeatherMap 5-day / 3-hour forecast JSON

2.4 Services

2.4.1 AuthService

Handles registration, login, OTP verification, and OTP resend. Key design decisions:

- OTP generation uses java.security.SecureRandom (6-digit, zero-padded) instead of Math.random().
- Email is sent @Async — the HTTP response is never blocked by SMTP.
- OTP upsert: existing token for user is deleted before saving the new one.
- OTP TTL: 5 minutes (300,000 ms).
- All error paths throw typed ApiException — no raw RuntimeException.
- @Transactional on all write operations.

2.4.2 JwtService

Issues and validates HMAC-SHA JWTs. Configuration:

- Secret: injected from jwt.secret property (minimum 256 bits recommended).
- Expiry: injected from jwt.expiry-ms (default 604,800,000 ms = 7 days).
- Payload: subject = userId (Long as string), claim email = user email.
- Validation: JwtException or IllegalArgumentException → isValid() returns false.

2.4.3 PlantService

Orchestrates the complete upload-and-analyze pipeline. Steps:

 © Harvest Box — All Rights Reserved

- Calls ImageStorageService.save() to upload to S3.
- Builds a payload map including optional fieldName, lat, lon.
- Calls MLClientService.analyzeFieldWithPayload() to hit the ML service.
- Maps ML response fields (prediction, confidence, severity_num, fusionMethod, weather.disease_risk) onto the Plant entity.
- Persists the Plant entity to the database.
- Returns PlantResponse DTO with human-readable severityLabel.
- Implements result caching via @Cacheable on getReports() and @CacheEvict on writes (Spring Cache, Simple provider).

2.4.4 ImageStorageService

AWS S3 upload with input validation:

- Allowed MIME types: image/jpeg, image/png, image/webp.
- Max file size: 5 MB.
- S3 key pattern: uploads/{userId}/{UUID}_{sanitized_filename}.
- @PostConstruct verifies bucket existence (never creates it — the IAM policy restricts CreateBucket).
- Returns the public S3 URL: https://{bucket}.s3.{region}.amazonaws.com/{key}.

2.4.5 MLClientService

HTTP client for the FastAPI ML service. Features:

- analyzeFieldWithPayload(Map) — accepts a pre-built payload supporting optional lat, lon, fieldName.
- Distinct error mapping: 422 Unprocessable Entity → ApiException.unprocessableEntity; 400 Bad Request → ApiException.badRequest; 5xx → ApiException.internalServerError.
- ML service URL configured via ml.base-url property (default http://localhost:8000).

2.4.6 EmailService

Sends verification emails via Spring Mail. Marked @Async so it never blocks registration. Failures are logged but do not abort the registration flow. Email subject: 'Your Harvest Box Verification Code'. OTP included in plain text body.

2.4.7 UserService

User profile business logic. getSummary() queries PlantRepository for total/healthy/warning counts per user. updateProfile() allows updating name and location (email and password are immutable through this endpoint).

2.5 Security Configuration

Spring Security is configured as a stateless JWT filter chain. Key aspects:

- CSRF disabled (stateless API — no session).
- Session creation policy: STATELESS.

 © Harvest Box — All Rights Reserved

- OPTIONS preflight requests are explicitly permitted before the JWT filter runs. Without this, Spring Security intercepts CORS preflight and returns 401, causing the browser to report a CORS error on all cross-origin requests.
- Public paths: /api/v1/auth/**, /h2-console/**, /swagger-ui/**, /api-docs/**, /actuator/health.
- All other paths require a valid JWT.
- Custom AuthenticationEntryPoint returns structured JSON 401 with timestamp, status, error, message, and path.

AuthFilter (OncePerRequestFilter):

- Reads Authorization: Bearer <token> header.
- Calls JwtService.isValid() — invalid/missing token falls through to the security chain.
- Sets UsernamePasswordAuthenticationToken in SecurityContextHolder with userId as the principal.
- Controllers read the userId via auth.getPrincipal().

2.6 Database Migrations (Flyway)

V1__init_schema.sql — Initial schema:

- users table with email UNIQUE index.
- verification_tokens table with FK to users (ON DELETE CASCADE).
- plant table with FK to users (ON DELETE CASCADE), indexes on user_id and analyzed_at DESC.

V2__add_hybrid_ml_columns.sql — Hybrid ML enrichment:

- Adds confidence DOUBLE PRECISION DEFAULT 0.0 to plant.
- Adds fusion_method VARCHAR(50) DEFAULT 'local_only' to plant.
- Adds field_name VARCHAR(255) DEFAULT '' to plant.
- Adds weather_risk VARCHAR(20) DEFAULT 'Unknown' to plant.
- All columns nullable — existing rows are unaffected.

2.7 Configuration Profiles

Profile	Database	Activation	Notes
(base)	—	Always loaded	Port 8080, multipart 5MB, Swagger enabled, graceful shutdown 20s
dev	H2 in-memory	SPRING_PROFILES_ACTIVE=dev	H2 console at /h2-console, SQL logging, devtools
prod	PostgreSQL	SPRING_PROFILES_ACTIVE=prod	Full PostgreSQL URL from env var, Flyway auto-migrate

 © Harvest Box — All Rights Reserved

3. ML Service (FastAPI)

The ML service is a Python 3.11+ FastAPI microservice responsible for image inference, weather integration, LLM report generation, PDF rendering, and S3 storage. It is consumed exclusively by the backend; the frontend never calls it directly.

3.1 Technology Stack

Component	Technology	Version
Framework	FastAPI	>=0.111.0
ASGI Server	Uvicorn	>=0.29.0
Deep Learning	PyTorch	>=2.2.0
Vision	TorchVision + Pillow	>=0.17.0 / >=10.0
Model Architecture	DenseNet121	Custom head (4 classes)
PDF Generation	fpdf2 + Matplotlib	>=2.7.9 / ==3.9.0
Cloud Storage	boto3	>=1.34.0
Weather API	Open-Meteo (requests)	No API key required
Report LLM (primary)	Google Gemini 2.5 Flash	ENABLE_GEMINI=true
Report LLM (fallback)	Qwen 2.5 72B via HuggingFace	Default
Config	python-dotenv + pydantic	>=1.0.0 / >=2.6.0

3.2 Application Bootstrap (app.py)

FastAPI application with an async lifespan context manager:

- Startup: validates AWS credentials are present in settings; validates S3_BUCKET is set; calls load_model() to load best_grape_model.pth into memory.
- Shutdown: logs service shutdown.
- Middleware: per-request UUID injection (X-Request-ID header), response timing (X-Response-Time header).
- Global exception handler: catches all unhandled exceptions and returns structured JSON with detail and requestId fields.
- CORS: configured from CORS_ORIGINS env var (default *).
- Routes registered at prefix /api/v1: health_route (health.py) and analyze (analyze.py).

3.3 Configuration (config/settings.py)

All environment variables are centralised in a single Settings class loaded at import time from the .env file. Key settings:

 © Harvest Box — All Rights Reserved

Variable	Default	Purpose
AWS_ACCESS_KEY_ID	(required)	S3 authentication
AWS_SECRET_ACCESS_KEY	(required)	S3 authentication
AWS_DEFAULT_REGION	ap-south-1	S3 region
S3_BUCKET	harvest-box-bucket	Bucket for images and PDFs
GEMINI_API_KEY	(empty)	Google Gemini report generation
HF_API_KEY	(empty)	HuggingFace Qwen 72B fallback
ENABLE_GEMINI	false	Toggle hybrid Gemini vision + report
ENABLE_FORECAST	true	Persist scan history for trend charts
DEFAULT_LAT	12.9716 (Bengaluru)	Fallback lat when not supplied per-request
DEFAULT_LON	77.5946 (Bengaluru)	Fallback lon when not supplied per-request
MODEL_PATH	model/ best_grape_model.pth	Local model file
MODEL_VERSION	v1	Version label in responses
ENV	development	Reported in /health
LOG_LEVEL	INFO	Logging verbosity

3.4 Analysis Pipeline (routes/v1/analyze.py)

POST /api/v1/analyze-field — the core orchestration endpoint. Accepts a FieldRequest Pydantic model and executes 9 sequential steps:

Step 1 — Per-image inference:

- Downloads each image from S3 via storage.download_image().
- Calls services/inference.predict_image() to get prediction, confidence, and class probabilities.
- Attaches row, col, image_url to each result dict.

Step 2 — Multi-image aggregation:

- Calls services/aggregation.aggregate_predictions() for confidence-weighted voting when multiple images are submitted.
- For single images, passes through the result with all hybrid keys.

Step 3 — Heatmap:

- Calls services/heatmap.build_heatmap() to produce a {row,col: {disease, confidence, severity}} map.
- Extracts severity string (none / medium / high) for the current cell.
- Converts severity to float: none=0.0, medium=0.5, high=1.0.

Step 4 — Weather:

 © Harvest Box — All Rights Reserved

- Calls services/weather.get_current_weather(lat, lon) using caller-supplied coordinates or settings.DEFAULT_LAT/LON.
- Returns temperature, humidity, precipitation, wind speed, WMO weather code, disease_risk (Low/Moderate/High), and risk_reason.

Step 5 — Forecast store:

- Calls services/forecast_store.save_result() to append an analysis record to forecast_data/analysis_history.json.

Step 6 — Historical summary:

- Calls services/forecast_store.get_field_summary() to retrieve aggregate disease counts and per-cell history for the user.

Step 7 — Report generation:

- Calls services/report.generate_report_text() to produce a 9-section AI-authored agronomic report.
- Passes scan results, weather, forecast summary, and field name to the LLM.

Step 8 — PDF generation and S3 upload:

- Calls services/pdf.generate_pdf() to create a multi-chart PDF at reports/{user_id}_{timestamp}.pdf.
- Calls services/storage.upload_pdf() to upload the PDF to S3 under key reports/{userId}/field_r{row}_c{col}_{UUID}.pdf.

Step 9 — Flat JSON response:

- Returns a flat dict (not nested) so the Java backend can read top-level keys directly with Map.get('pdf_url') etc.
- Critical contract keys: pdf_url, prediction, confidence, severity, severity_num, heatmap.

3.5 Inference Service (services/inference.py)

DenseNet121-based classifier. Architecture:

- Base: models.densenet121(weights=None).
- Classifier head replaced: Linear(in_features, 4) for 4 disease classes.
- Loaded from best_grape_model.pth via torch.load(map_location='cpu').
- Model set to eval() mode at load time.
- Thread-safe: protected by threading.Lock during load and inference.

Input transform pipeline:

- Resize(256) → CenterCrop(224) → ToTensor() → Normalize(ImageNet mean/std).

Output:

- Softmax applied to logits.
- Winner = argmax(probs).
- Confidence = probs[winner] * 100, rounded to 2 decimal places.
- all_probs = {class: prob * 100} for all 4 classes.
- fusion_method always 'local_only' in current production build.

Class	Severity	Description
Healthy	none	No disease detected

 © Harvest Box — All Rights Reserved

Black_Rot	high	Guignardia bidwellii fungal infection
Downy_Mildew	medium	Plasmopara viticola oomycete infection
Powdery_Mildew	medium	Erysiphe necator fungal infection

3.6 Weather Service (services/weather.py)

Fetches real-time weather from the Open-Meteo API (no API key required). Features:

- 30-minute in-memory cache keyed by (lat, lon) rounded to 3 decimal places. Cache is protected by threading.Lock.
- 10-second HTTP timeout.
- Fetches: temperature_2m, relative_humidity_2m, precipitation, weather_code, wind_speed_10m.
- WMO code decoded to human-readable string from a 25-entry lookup table.

Disease risk heuristic:

- High: humidity >= 85% AND 15°C <= temp <= 30°C (wet+warm = ideal for Downy Mildew / Black Rot).
- Moderate: humidity >= 70% AND 10°C <= temp <= 35°C (scouting warranted).
- Low: all other conditions.
- Returns risk_reason string explaining the assessment in agricultural terms.

3.7 Report Generation (services/report.py)

Generates structured, professional agronomic reports using LLMs. Architecture:

LLM selection:

- ENABLE_GEMINI=true → Gemini 2.5 Flash (google-generativeai), falls back to HuggingFace on error.
- ENABLE_GEMINI=false → Qwen 2.5 72B via HuggingFace Inference Router (default in production).

Prompt construction (_build_prompt()):

- Scan findings section: row, col, prediction, confidence for each grid cell.
- Dominant disease: highest-confidence prediction across all cells.
- ASCII confidence bar chart embedded in the prompt (20-column blocks, 5% per block).
- Current weather block: temperature, humidity, rainfall, wind speed, disease risk.
- Historical scan data: disease counts from forecast_store.

Report structure (9 sections):

Section	Contents
1. Executive Summary	Field health score (x/10), 2-3 sentence overview, single critical action
2. Disease Diagnosis	Disease name, scientific name, causal agent, symptom progression
3. Weather-Disease Interaction	How current humidity/temp/rain amplifies disease; 7-day risk chart
4. Forecast & Progression	Scenario A (no treatment) vs Scenario B (with treatment); yield loss/save %

 © Harvest Box — All Rights Reserved

5. Treatment Plan	Immediate chemical (Indian brands, dosage), organic alternatives
6. Field Spatial Analysis	Affected grid coordinates, spread direction, quarantine buffer zones
7. Model Confidence	Confidence % with ASCII probability distribution chart
8. Monitoring Schedule	Rescan interval, early-warning signs
9. Long-Term Prevention	Variety selection, IPM calendar, soil health, crop rotation

HuggingFace retry logic: 3 attempts with exponential delay (2s, 4s). Returns error string on total failure.

3.8 PDF Generation (services/pdf.py)

Creates multi-page PDF reports using fpdf2 and matplotlib. All text is sanitized through _safe() which strips markdown, normalises unicode punctuation, and encodes to latin-1. PDF sections:

Cover / Header: Green banner (34,139,34) with title, field name, and generation timestamp.

Weather Strip: Compact strip showing current conditions and colour-coded disease pressure (green/orange/red).

AI Field Summary: Full LLM report text rendered line-by-line; uppercase lines rendered as bold section headings.

Visual Analysis (3 charts via matplotlib):

- Figure 1 — Grouped horizontal bar chart: class probability per grid cell.
- Figure 2 — Pie chart: disease distribution across all analysed cells.
- Figure 3 — Historical trend line: infected vs total scanned cells over time (only when forecast history exists).

Forecast & Historical Context: Aggregate scan counts, disease breakdown, infected cell list.
Grid Analysis Table: Per-cell table showing row, col, disease, confidence %, severity, and fusion method.
Class Probabilities: Inline bar rendering (fpdf2 native) for up to 2 cells.
Footer: AI model info, user ID, generation datetime.

3.9 Storage Service (services/storage.py)

AWS S3 helper using boto3. Key behaviours:

- Lazy singleton client: created on first call, reused thereafter.
- Credentials: explicit when set in settings, otherwise falls back to boto3's default chain (IAM role on EC2/ECS).
- download_image(url): extracts S3 key from virtual-hosted or path-style URLs, calls get_object().
- upload_pdf(local_path, key): uploads local file with ContentType application/pdf, returns public S3 URL.

3.10 Aggregation Service (services/aggregation.py)

Confidence-weighted voting for multi-image submissions:

 © Harvest Box — All Rights Reserved

- Single image: pass-through.
- Multiple images: sums confidence scores per predicted class. Winner = class with highest total confidence.
- Returned confidence = mean confidence of all images that predicted the winner.

3.11 Heatmap Service (services/heatmap.py)

Builds a coordinate-keyed heatmap dict from ML predictions. Keys are 'row,col' strings. Each cell contains disease name, confidence, and severity mapped via SEVERITY_MAP: Healthy='none', Black_Rot='high', Downy_Mildew='medium', Powdery_Mildew='medium'.

3.12 Forecast Store (services/forecast_store.py)

Per-user analysis history persisted to forecast_data/analysis_history.json. Thread-safe via threading.Lock. Functions:

- save_result(): appends a record with timestamp, user_id, field_name, row, col, prediction, confidence, severity, fusion_method, all_probs.
- get_history(user_id, limit=200): returns the most recent limit records for a user.
- get_cell_history(user_id, row, col, limit=50): history for a specific grid cell.
- get_field_summary(user_id): aggregate statistics — total_analyses, disease_counts dict, cell_disease_map, latest_by_cell, trend (last 30 records).

3.13 Middleware (middleware/error_handler.py)

Global exception handler returning flat JSON (not envelope-wrapped): {detail, requestId}. Returns HTTP 500. Java backend reads the detail field for error messages.

3.14 Utilities

utils/logger.py — Structured logger: single StreamHandler to stdout, format: timestamp | LEVEL | message. Log level controlled by LOG_LEVEL env var.

utils/response.py — success(data) and error(message) envelope helpers used by health.py and admin endpoints. NOTE: analyze_field does NOT use these — it returns a flat dict.

 © Harvest Box — All Rights Reserved

4. Frontend Application (React)

The frontend is a React 19 Single Page Application built with Vite 7. It provides the complete farmer-facing interface: authentication, live weather dashboard, field setup (image upload), analytics/reports, and user profile.

4.1 Technology Stack

Dependency	Version	Purpose
React	^19.2.0	Core UI framework
React Router DOM	^7.12.0	Client-side routing
Zustand	^4.5.4	Global state management
Recharts	^2.12.7	Bar, line, and pie charts in analytics
DOMPurify	^3.1.6	XSS sanitization for user input
Vite	^7.2.4	Build tool and dev server

4.2 Application Entry (App.jsx)

The root component sets up the entire application shell:

- BrowserRouter wraps all routing.
- AuthProvider wraps all routes providing auth context.
- Lazy loading: Dashboard, FieldSetup, Reports, and User pages are React.lazy() — code-split for performance.
- Protected routes: all pages except /login and /register are wrapped in ProtectedRoute.
- ErrorBoundary wraps all lazy-loaded pages.
- Navbar is suppressed on /login and /register.

4.3 API Client (src/api/client.js)

Centralised fetch wrapper with JWT injection, error handling, and file upload support:

apiFetch(endpoint, options):

- Reads auth_token from localStorage and injects as Authorization: Bearer header.
- Parses JSON response; falls back to plain text.
- On 401: clears auth_token, redirects to /login (no alert shown — silent redirect).
- On error: throws ApiError(message, status, data) — error message extracted from data.message, data.error, or data.errors[0].

apiUpload(endpoint, formData):

 © Harvest Box — All Rights Reserved

- Same as apiFetch but sends FormData (no Content-Type header — let the browser set multipart boundary).
- Used exclusively by FieldSetup for image upload.

authApi object:

- login(email, password), register(name, email, password), verifyOtp(email, otp), resendOtp(email) — convenience wrappers.

4.4 Global State (src/store/useAppStore.js)

Three Zustand stores:

useReportsStore:

State / Action	Type	Purpose
reports	Array	All analysis reports for current user
loading / error	Boolean / String	Loading and error states
lastFetched	Timestamp	Cache invalidation
setReports / addReport	Actions	Update reports list
isStale()	() => boolean	Returns true if > 60 seconds since last fetch
clearCache()	Action	Force refetch on next render

useUploadStore:

State / Action	Type	Purpose
status	String	idle | uploading | processing | generating | done | error
progress	Number 0-100	Progress bar value
message	String	User-facing status message
result	Object | null	{pdfUrl, health} on success
start / setProcessing / setGenerating / setDone / setError / reset	Actions	State machine transitions

useForecastStore:

State	Purpose
data	Normalised {current, forecast, coords} weather object

 © Harvest Box — All Rights Reserved

loading / error	Weather fetch states
isStale()	Returns true if > 10 minutes since last fetch

4.5 Authentication (src/context/)

- AuthContext.jsx: creates the React context (null default).
- AuthProvider.jsx: wraps the app, provides the auth state (user, token, login, logout).
- useAuth.js: custom hook — useContext(AuthContext).
- ProtectedRoute.jsx: reads auth state; redirects to /login if not authenticated.

4.6 Pages

4.6.1 Auth Page (src/pages/Auth/Auth.jsx)

Handles both /login and /register routes on a single component. Three states: login form, register form, OTP verification form. OTP input uses the OtpInput component for digit-by-digit entry. On successful login/verify-otp, the JWT is stored in localStorage and the user is redirected to /. Resend OTP button available with cooldown.

4.6.2 Dashboard Page (src/pages/Dashboard/Dashboard.jsx)

The main landing page for authenticated users. Components:

- Greeting: time-aware (Good morning / afternoon / evening).
- Weather Hero card: current temperature, description, location, humidity, wind, feels-like, pressure, sunrise/sunset.
- Scan Trends card: fetches last 10 reports; shows healthy/diseased/avg-confidence pills, disease breakdown progress bars, and a 5-item recent scan list.
- Forecast Strip: 7 day buttons with high/low temps and rain probability.
- Day Detail card: detailed stats for selected forecast day.
- Metric Cards (×4): Temperature, Humidity, Wind Speed, Feels Like — each with a sparkline SVG.
- Forecast Trend Chart: custom SVG line chart with gradient area fill; tabs for temp/humidity/wind.
- Rain Probability chart: vertical bars coloured by level (green/yellow/red).
- Smart Field Alerts: combines weather conditions (rain, wind, humidity, heat) + plant scan data (diseased scans, high weather risk) into actionable alert rows.
- Skeleton loading state while weather is fetching.

4.6.3 Field Setup Page (src/pages/FieldSetup/FieldSetup.jsx)

Leaf image upload and analysis trigger. Features:

- LeafInstructions accordion: 6 photography rules (plain background, one leaf, fill frame, top surface, good lighting, pick symptomatic leaf).
- GPS capture: navigator.geolocation on mount (eagerly, before upload) with IP fallback. Coords attached to upload payload for accurate weather.
- Drag-and-drop dropzone + file input. Client-side validation: images only, max 5 MB.
- Preview grid with remove button for each image.
- Fields: Field Area Name (required), Row (required), Column (required).

 © Harvest Box — All Rights Reserved

- Upload flow: simulated processing states (uploading 10%, processing 45%, generating 80%, done 100%) with actual upload running in parallel.
- UploadProgress component shows animated progress bar and status message.
- On success: shows health status + 'View Full PDF Report' button.
- 'Analyze another' button resets entire form.

4.6.4 Reports / Analytics Page (src/pages/Reports/Reports.jsx)

Paginated report browser and analytics dashboard. Two tabs:

List tab:

- Searchable report list by field name, disease, grid position.
- Detail panel on selection: scan ID, health badge, weather risk badge, meta grid (field, grid, diagnosis, severity, confidence, AI model), disease severity gauge, model confidence bar, weather pressure bar.
- PDF report link or 'not available' notice.

Analytics tab:

- Summary cards: Total Scans, Healthy, Diseased (with %), Avg Confidence, High Weather Risk count.
- Plant Confidence Ratings: Recharts BarChart, one bar per scan coloured by disease, custom tooltip.
- Disease Distribution: Recharts PieChart (donut) with list breakdown.
- Confidence Trend: Recharts LineChart with dual lines (confidence + severity over time).

Toast notification system (2s duration): shows model scope warning on first visit.
Refresh button clears Zustand cache and forces refetch.

4.6.5 User Profile Page (src/pages/User/User.jsx)

Displays and allows editing of user profile (name, location). Shows summary stats from /api/v1/user/summary.

4.7 Components

Component	File	Description
Navbar	components/Nav_Bar/Nav_Bar.jsx	Navigation bar with active route highlighting
ProtectedRoute	components/ProtectedRoute.jsx	Wraps protected routes; redirects unauthenticated users
ErrorBoundary	components/ErrorBoundary/ErrorBoundary.jsx	Catches React render errors; shows fallback UI with optional retry
OtpInput	components/OtpInput.jsx	6-digit OTP input with auto-focus-next-field behaviour
UploadProgress	components/UploadProgress/UploadProgress.jsx	Animated progress bar reading from useUploadStore

 © Harvest Box — All Rights Reserved

RiskGauge	components/Charts/RiskGauge.jsx	SVG semi-circular gauge for disease severity (0-100)
ConfidenceBar	components/Charts/ ConfidenceBar.jsx	Horizontal progress bar for model confidence
LineChart	components/Charts/LineChart.jsx	Recharts-based line chart wrapper
ConfidenceBar	components/Charts/ ConfidenceBar.jsx	Animated confidence bar with colour coding
ChatBot	components/ChatBot/ChatBot.jsx	Prototype farmer assistant chatbot (calls localhost:5000/chat)
Icon	components/Icon/Icon.jsx	Harvest Box SVG logo

4.8 Weather Service (src/services/weatherService.js)

Location strategy: GPS (navigator.geolocation) with 10s timeout → IP fallback (ipapi.co).
Functions:

- getCoords(): resolves {lat, lon} — exported for use by FieldSetup.
- getWeatherAndForecast(): calls both /api/v1/weather and /api/v1/weather/forecast through the backend proxy, normalises responses.

Normalisation:

- normaliseCurrentWeather(): extracts temperature, humidity, windSpeed, windDeg, description, icon, high, low, feelsLike, pressure, visibility, sunrise, sunset.
- normaliseForecast(): groups OWM 3-hour entries by date, uses 12:00 entry for condition/icon, computes daily hi/lo/avg humidity/avg wind/avg rain probability.

 © Harvest Box — All Rights Reserved

5. Deployment

5.1 Docker — Backend

The backend includes a Dockerfile and docker-compose.yml for containerised deployment.

- Base image: Eclipse Temurin 21 (slim).
- docker-compose.yml: backend service with environment variable passthrough.
- docker-compose.prod.yml: production override with PostgreSQL service and Nginx reverse proxy.

5.2 Docker — ML Service

The ML service includes a Dockerfile and start.sh script:

- Installs requirements.txt.
- Copies model file and source.
- start.sh: runs uvicorn on 0.0.0.0:8000.
- .dockerignore: excludes __pycache__, .env, reports/, forecast_data/.

5.3 Nginx Configuration

nginx/nginx.conf (included with backend) acts as a reverse proxy in production:

- Listens on port 80.
- Proxies all /api/ requests to the Spring Boot container.
- Serves static files for the frontend build output.

5.4 CI/CD (GitHub Actions)

.github/workflows/ci.yml defines the CI pipeline:

- Triggers on push and pull_request to main.
- Java backend: ./mvnw test
- Frontend: npm ci && npm run build

5.5 Frontend Deployment (Vercel)

vercel.json configures the React SPA for Vercel:

- rewrites: all paths → /index.html (client-side routing).
- VITE_API_BASE_URL must be set in Vercel project environment variables.

5.6 Backend Deployment (Render)

Dockerfile-based deployment on Render. Required environment variables:
SPRING_PROFILES_ACTIVE=prod, DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET,

 © Harvest Box — All Rights Reserved

AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY, S3_BUCKET, MAIL_HOST, MAIL_USERNAME, MAIL_PASSWORD, WEATHER_API_KEY, ML_BASE_URL.

 © Harvest Box — All Rights Reserved

6. Security Architecture

6.1 Authentication Flow

- Registration: user submits email + password + name. BCrypt encodes the password. A 6-digit OTP (SecureRandom) is generated and emailed. JWT is only issued after OTP verification.
- Login: password compared with BCrypt. Verified flag checked. JWT issued with userId + email claims.
- All subsequent requests carry JWT in Authorization: Bearer header. AuthFilter validates the token and sets the userId as the Spring Security principal.

6.2 JWT

- Algorithm: HMAC-SHA (via Keys.hmacShaKeyFor — recommended 256+ bit secret).
- Expiry: 7 days by default (configurable).
- Payload: subject = userId, claim email. No sensitive data (no password hash, no roles).

6.3 Input Validation

- Backend: @Valid annotations on all request bodies. GlobalExceptionHandler catches MethodArgumentNotValidException and returns 400 with field errors.
- ML Service: Pydantic models validate all incoming request fields.
- Frontend: client-side validation in FieldSetup (file type, file size, required fields). DOMPurify sanitizes field name input.
- Image validation (backend): MIME type whitelist (JPEG/PNG/WEBP), 5 MB size cap, filename sanitization (replaces non-alphanumeric chars).

6.4 Rate Limiting

Bucket4j filter (RateLimitFilter) limits API requests per IP address. Configured as a Spring Bean in AppConfig.

6.5 CORS

CorsConfig permits cross-origin requests from the Vercel frontend domain (configured via cors.allowed-origins property). SecurityConfig delegates CORS to CorsConfig — no duplicate source.

 © Harvest Box — All Rights Reserved

7. Testing

7.1 Backend Tests

Test Class	Type	Coverage
ProjectApplicationTests	Spring context load	Verifies application context starts correctly
AuthServiceTest	Unit (Mockito)	Registration, login, OTP verify, resend — all success and failure paths
PlantServiceTest	Unit (Mockito)	uploadAndAnalyze, getReports, create — mocks S3 and ML client
AuthControllerIntegrationTest	Spring MockMvc integration	Full HTTP request/response for all /auth/* endpoints

Run all tests: ./mvnw test

7.2 ML Service Tests

pytest is configured in requirements.txt with pytest-asyncio and httpx for async endpoint testing. Test files are placed alongside source modules. Run: pytest

 © Harvest Box — All Rights Reserved

8. Complete File Reference

8.1 Backend Files

File	Purpose
pom.xml	Maven build file. Java 21, Spring Boot 3.2.1 parent. Dependencies: web, JPA, validation, security, actuator, PostgreSQL, H2, Flyway, S3, mail, JWT (JJWT 0.12.3), Jackson JSR310, SpringDoc 2.3.0, Bucket4j 8.7.0, cache, Lombok, MapStruct, devtools. Compiler plugin configures annotation processors: Lombok → lombok-mapstruct-binding → MapStruct.
ProjectApplication.java	Spring Boot entry point. @SpringBootApplication, main() calls SpringApplication.run().
config/AppConfig.java	Defines RestTemplate @Bean (used by MLClientService and WeatherController).
config/AuthFilter.java	OncePerRequestFilter. Reads Bearer token, validates via JwtService, sets SecurityContextHolder principal to userId (Long).
config/CorsConfig.java	Registers CorsConfigurationSource with allowed origins from cors.allowed-origins property.
config/JacksonConfig.java	Registers JavaTimeModule for LocalDateTime serialization.
config/OpenApiConfig.java	Configures SpringDoc with bearerAuth security scheme.
config/RateLimitFilter.java	Bucket4j per-IP rate limiter registered as OncePerRequestFilter.
config/RequestLoggingFilter.java	Logs each request method, path, status, and duration.
config/S3Config.java	Creates S3Client @Bean with explicit AWS credentials from properties.
config/SecurityConfig.java	Stateless security chain: CSRF off, CORS enabled, OPTIONS permitted, auth endpoints public, all others require JWT, custom 401 JSON entry point.
controller/AuthController.java	POST /api/v1/auth/register|login|verify-otp|resend-otp. All public. Delegates to AuthService.
controller/ImageController.java	POST /api/v1/images/upload. JWT required. Accepts file, row, col, fieldName, lat, lon. Delegates to PlantService.uploadAndAnalyze().
controller/PlantController.java	GET|POST /api/v1/plants. JWT required. Paginated reports and manual creation.
controller/UserController.java	GET /api/v1/user/summary and PUT /api/v1/user/profile. JWT required.
controller/WeatherController.java	GET /api/v1/weather and /api/v1/weather/forecast. JWT required. Proxies OpenWeatherMap — API key server-side only.
dto/AuthResponse.java	Response: {UserDto user, String message, String token}. UserDto: {id, name, email}.
dto/LoginRequest.java	Request: {String email, String password} with @NotBlank/@Email validation.
dto/RegisterRequest.java	Request: {String name, String email, String password} with validation.
dto/MLAnalysisResponse.java	Maps ML service response: prediction, confidence, severity, pdfUrl, etc.

 © Harvest Box — All Rights Reserved

dto/PlantResponse.java	Public DTO for Plant entity: id, rowIndex, colIndex, health, severity, severityLabel, confidence, fusionMethod, fieldName, weatherRisk, reportPdfUrl, analyzedAt (ISO string).
exception/ApiException.java	RuntimeException with HTTP status. Factory methods: badRequest(400), unauthorized(401), notFound(404), conflict(409), unprocessableEntity(422), internalServerError(500).
exception/ GlobalExceptionHandler.java	@ControllerAdvice catching ApiException, MethodArgumentNotValidException, and generic Exception.
model/User.java	JPA entity: id, email (UNIQUE), passwordHash, name, location, verified, createdAt. Index on email.
model/Plant.java	JPA entity: id, userId, rowIndex, colIndex, imageUrl, health, severity, confidence, fusionMethod, fieldName, weatherRisk, reportPdfUrl, analyzedAt. Indexes on userId and analyzedAt DESC.
model/VerificationToken.java	JPA entity: id, userId, otp, expiryDate. Index on userId.
repository/UserRepository.java	JPA repo: findByEmail, existsByEmail.
repository/PlantRepository.java	JPA repo: findByUserId (Pageable), countByUserId, countByUserIdAndHealthIgnoreCase.
repository/ VerificationTokenRepository.java	JPA repo: findByUserId, deleteByUserId.
service/AuthService.java	register(), login(), verifyOtp(), resendOtp(). SecureRandom OTP. @Async email. @Transactional writes.
service/JwtService.java	generate(userId, email), extractUserId(token), extractEmail(token), isValid(token). HMAC-SHA, config-driven secret and expiry.
service/PlantService.java	uploadAndAnalyze(), getReports() @Cacheable, create() @CacheEvict. Orchestrates S3 upload + ML call + DB save.
service/ImageStorageService.java	save(file, userId): validates MIME type and size, uploads to S3, returns URL. @PostConstruct bucket verification.
service/MLClientService.java	analyzeFieldWithPayload(Map): POST to ML service. analyzeField() wrapper for backward compatibility. Typed error mapping.
service/EmailService.java	@Async sendVerificationEmail(). Spring JavaMailSender. Silent failure (log only).
service/UserService.java	getSummary(userId): plant counts + profile. updateProfile(userId, body): name + location.
resources/application.yml	Base config: app name, multipart 5MB, cache simple, JPA no open-in-view, port 8080, graceful shutdown, compression, Swagger paths, AWS/S3 config.
resources/application-dev.yml	H2 datasource, H2 console, SQL logging, devtools.
resources/application-prod.yml	PostgreSQL datasource from env vars, Flyway validate, SQL logging off.
resources/db/migration/ V1__init_schema.sql	Creates users, verification_tokens, plant tables with indexes.
resources/db/migration/ V2__add_hybrid_ml_columns.sql	Adds confidence, fusion_method, field_name, weather_risk to plant.
resources/logback-spring.xml	Logback config: rolling file appender in prod, console in dev.

 © Harvest Box — All Rights Reserved

Dockerfile	Builds Spring Boot fat JAR, exposes port 8080, sets JAVA_OPTS.
docker-compose.yml	Backend service with env vars.
docker-compose.prod.yml	Backend + PostgreSQL + Nginx.
nginx/nginx.conf	Reverse proxy: /api/ → Spring Boot, static files → React build.
.github/workflows/ci.yml	GitHub Actions CI: mvn test + npm build on push/PR.

8.2 ML Service Files

File	Purpose
app.py	FastAPI app entry point. Lifespan: credential check + load_model(). Middleware: request ID, timing. CORS. Routes registered at /api/v1.
config/settings.py	Settings class loading all env vars from .env. Single source of truth for configuration.
routes/v1/analyze.py	POST /api/v1/analyze-field. 9-step pipeline: inference → aggregation → heatmap → weather → forecast save → forecast load → report gen → PDF → S3 upload → flat JSON response.
routes/v1/health.py	GET /api/v1/health: returns {status, version, env, features{gemini, forecast}}.
services/inference.py	DenseNet121 loader and predictor. Thread-safe. ImageNet normalisation. Softmax output. Returns prediction, confidence, all_probs, fusion_method.
services/weather.py	Open-Meteo client. 30-min in-memory cache. Disease risk heuristic. WMO code decoder.
services/report.py	LLM report generator. Prompt factory. Gemini 2.5 Flash + HF Qwen 72B fallback. Retry logic. 9-section structured prompt. ASCII confidence bar chart.
services/pdf.py	fpdf2 + matplotlib PDF generator. Cover, weather strip, AI text, 3 charts, grid table, class probability bars, footer.
services/storage.py	boto3 S3 client. download_image() (virtual-hosted and path-style URL parsing). upload_pdf() with ContentType.
services/aggregation.py	Confidence-weighted voting for multi-image submissions.
services/heatmap.py	Coordinate-keyed heatmap builder. Severity mapping per disease class.
services/forecast_store.py	Thread-safe JSON persistence for per-user analysis history. save_result(), get_history(), get_cell_history(), get_field_summary().
middleware/error_handler.py	Global exception handler returning {detail, requestId} JSON with HTTP 500.
utils/logger.py	Structured StreamHandler logger. Level from LOG_LEVEL env var.
utils/response.py	success() and error() envelope helpers for non-analysis endpoints.
model/best_grape_model.pth	Fine-tuned DenseNet121 weights. Classes: Black_Rot, Downy_Mildew, Healthy, Powdery_Mildew.
forecast_data/analysis_history.json	Per-user scan history (auto-created). JSON array of analysis records.

 © Harvest Box — All Rights Reserved

requirements.txt	Python dependencies: fastapi, uvicorn, torch, torchvision, Pillow, numpy, opencv-python-headless, matplotlib, boto3, requests, fpdf2, python-dotenv, pydantic, pytest, pytest-asyncio, httpx.
Dockerfile	Python 3.11 slim base. Installs requirements. Copies source and model. Runs start.sh.
start.sh	uvicorn app:app --host 0.0.0.0 --port 8000.
.dockerignore	Excludes __pycache__, .env, reports/, forecast_data/, model/.pth files.

8.3 Frontend Files

File	Purpose
src/main.jsx	React DOM render root. Mounts <App /> to #root.
src/App.jsx	Router setup, AuthProvider, lazy loading, ProtectedRoute, ErrorBoundary, Navbar suppression on auth pages.
src/api/client.js	apiFetch() and apiUpload() — JWT injection, 401 redirect, ApiError, authApi convenience methods.
src/context/AuthContext.jsx	Creates AuthContext (createContext(null)).
src/context/AuthProvider.jsx	Provides auth state: user, token, login(), logout(). Reads/writes auth_token in localStorage.
src/context/useAuth.js	useContext(AuthContext) hook.
src/store/useAppStore.js	Three Zustand stores: useReportsStore (60s cache), useUploadStore (upload state machine), useForecastStore (10-min weather cache).
src/services/weatherService.js	getCoords() (GPS + IP fallback), getWeatherAndForecast() (dual OWM proxy calls), normaliseCurrentWeather(), normaliseForecast().
src/pages/Auth/Auth.jsx	Login + register + OTP verification UI on a single route-aware component.
src/pages/Dashboard/Dashboard.jsx	Full weather dashboard: hero card, scan trends, forecast strip, day detail, 4 metric cards, forecast trend chart, rain probability, smart alerts.
src/pages/FieldSetup/FieldSetup.jsx	Leaf upload form: instructions accordion, GPS capture, drag-and-drop zone, preview grid, upload progress, result with PDF link.
src/pages/Reports/Reports.jsx	Report browser (list + detail) and analytics (summary cards, bar chart, pie chart, trend line). Search, filter, refresh, toast notifications.
src/pages/User/User.jsx	User profile view and edit.
src/components/Nav_Bar/Nav_Bar.jsx	Navigation bar with route-aware active state.
src/components/ProtectedRoute.jsx	Auth guard: redirects to /login if not authenticated.
src/components/ErrorBoundary/ErrorBoundary.jsx	React class error boundary with retry callback prop.
src/components/OtpInput.jsx	6-digit OTP input with auto-advance focus.

 © Harvest Box — All Rights Reserved

src/components/UploadProgress/UploadProgress.jsx	Progress bar and status message from useUploadStore.
src/components/Charts/RiskGauge.jsx	SVG semi-circular gauge for severity (0-100).
src/components/Charts/ConfidenceBar.jsx	Animated horizontal bar for confidence percentage.
src/components/Charts/LineChart.jsx	Recharts LineChart wrapper.
src/components/ChatBot/ChatBot.jsx	Prototype farmer chatbot (calls localhost:5000/chat — not production-integrated).
src/components/Icon/Icon.jsx	Harvest Box SVG brand icon.
src/styles/variables.css	CSS custom properties for the design system: colours, shadows, radii.
src/index.css	Global reset and base styles.
index.html	Vite HTML template with #root mount point.
vite.config.js	Vite configuration with @vitejs/plugin-react.
vercel.json	Vercel SPA routing: all paths → /index.html.
package.json	Dependencies: react 19, react-router-dom 7, zustand 4, recharts 2, dompurify. Dev: vite 7, eslint, @vitejs/plugin-react.

 © Harvest Box — All Rights Reserved

9. Glossary

Term	Definition
DenseNet121	Densely Connected Convolutional Network with 121 layers. Used as the fine-tuned image classification backbone.
Flyway	Database migration tool. Applies versioned SQL scripts (V1, V2...) to keep the schema in sync with the application code.
Fusion Method	How the final prediction was produced. 'local_only' = DenseNet121 alone. 'hybrid_geometric_mean' = weighted geometric mean of local + Gemini probabilities (future feature).
Weather Risk	Agronomic disease pressure assessment: Low / Moderate / High. Derived from temperature, humidity, and precipitation thresholds.
Confidence	Softmax probability of the winning class, expressed as a percentage (0-100).
Severity	Disease impact rating: none (0.0) / medium (0.5) / high (1.0). Mapped from prediction class via SEVERITY_MAP.
Forecast Store	JSON file (forecast_data/analysis_history.json) persisting per-user scan history for trend analysis.
Heatmap	Dictionary keyed by 'row,col' strings containing disease, confidence, and severity for each analysed grid cell.
WMO Code	World Meteorological Organisation weather code returned by Open-Meteo. Decoded to human-readable descriptions.
Bearer Token	JWT passed in the Authorization HTTP header as 'Bearer <token>'.
OTP	One-Time Password — 6-digit numeric code sent by email for account verification. Expires in 5 minutes.
S3 Key	Object path within the S3 bucket. Images: uploads/{userId}/{UUID}_{filename}. PDFs: reports/{userId}/field_r{row}_c{col}_{UUID}.pdf.

Harvest Box — Official Technical Documentation

Version 2.0.0 | Backend: Spring Boot 3.2 | ML: FastAPI + PyTorch | Frontend: React 19

© Harvest Box — All Rights Reserved. Confidential.